

Trust Lite v2 — Auditoría de Features y Plan de Transición

Documento de síntesis ejecutiva

Fecha: 2026-05-12

Sintetizador: writer (kanban task S1)

Fuentes: Auditorías A1–A7 (analyst, analyst-2, owl-alpha, owl-alpha-2, owl-alpha-3, researcher, writer)

Idioma: Español

Versión: 1.0

Resumen Ejecutivo

Trust Suite v1 es un sistema de gobernanza multi-árbol con 6 PWAs, economía interna compleja (berries, fiat ledger, fee protocol, escrow), pipeline democrático de necesidades (Need → Idea → Branch → Task con votación ponderada), parlamento de IAs (AI Council), y sistema de reputación con avales, golden tickets y skill influence. Tiene **~15,000 líneas de código** entre backend (Express + Prisma) y frontend (React + Vite).

Trust Maker (rama GitHub derivada de Trust Suite) es la destilación radical de ese sistema: un paperclip meritocrático open-source donde IAs entrenables compiten por tareas, ganan XP, y mejoran sus modelos vía fine-tuning. Sin gobernanza democrática, sin economía fiat interna, sin multi-PWA. Precio único dinámico sin tiers, orquestación autónoma vía Hermes Agent, y BYO API Keys para romper dependencia de big tech.

Hallazgos principales

Se auditaron **57 features** en 7 dominios:

Dominio	Features evaluadas	QUEDA	ARCHIVAR	ELIMINAR	% Código rescatable
Core Governance	7	4	0	1 (2 simplificar)	~40%
People & Skills	9	6	2	1	~70%
Economy & Finance	9	2	1	4 (2 reformular/ reemplazar)	~30%
AI & Automation	6	4	1	1	~65%
PWA Architecture	10	4	1	2 (3 absorber)	~80%
Analytics & Export	8	4	1	3	~55%
External/ Enterprise	8	0	2	6	~15%
TOTAL	57	24	8	18 (7 intermedios)	~50%

Conclusión central: Aproximadamente la mitad del código actual es rescatable directamente o con modificaciones menores. La otra mitad — el pipeline de gobernanza democrática, la economía fiat/berries, el multi-PWA, AI Council, y el pipeline enterprise — debe eliminarse. El código rescatable se concentra en: modelo de Trees, autenticación, sistema de XP/niveles, Career Path, motor económico de puntos, Hermes Bridge, y Concierge.

Cifras globales

- **Features evaluadas:** 57
- **QUEDA (rescatable directamente):** 24 (42%)
- **QUEDA con cambios (simplificar/absorber/reformular):** 7 (12%)

- **ARCHIVAR (código se conserva, no se activa):** 8 (14%)
 - **ELIMINAR (código a borrar):** 18 (32%)
 - **% global de código rescatable:** ~50–55%
 - **Líneas de código a eliminar estimadas:** ~7,000–8,000 LOC
 - **Modelos de schema a eliminar:** 14+
 - **Dependencias candidatas a eliminar:** pdfkit, recharts (parcial)
-

Decisiones de Leo (12 mayo 2026)

Decisiones ejecutivas que redefinen Trust Lite v2 como **Trust Maker**, una rama independiente de Trust Suite con modelo de negocio simplificado y orquestación autónoma vía Hermes Agent.

1. **Nombre:** Trust Maker (rama GitHub derivada de Trust Suite). Proyecto independiente con su propio roadmap, sin dependencia del mono-repo Trust Suite.
 2. **Precio único dinámico:** Sin tiers (Free/Pro/Cloud). Un solo precio calculado dinámicamente como: costos reales de IAs + árbol empresa transparente (costos operativos) + % crecimiento. Todos los usuarios pagan lo mismo.
 3. **Need Points:** 1000 para todos los árboles, distribuidos con sistema dinámico 1-10. Las IAs auto-priorizan las necesidades más apremiantes (problemas agregados) — sin votación humana.
 4. **Orquestación de IAs:** Hermes Agent maneja todo el ciclo de vida de las IAs (spawn, asignación de tasks, fine-tuning triggers, health checks, reciclaje). Un solo agente orquestador reemplaza el sistema de councils y cron jobs.
 5. **Servidor único:** Una instancia corriendo Trust Maker + Hermes Agent (2 aplicaciones, 1 servidor). Sin microservicios, sin colas distribuidas, sin multi-PWA.
 6. **BYO API Keys:** Usuarios pueden traer sus propias APIs de IAs (OpenAI, DeepSeek, Groq, etc.) → reciben descuento en la suscripción o se les paga por uso de sus keys. Esto rompe la dependencia de un proveedor central de inferencia y convierte a los usuarios en proveedores del ecosistema.
-

Tabla Maestra de Features

A continuación, las 57 features evaluadas con su dominio, decisión y justificación breve. Las decisiones se codifican como:

- **QUEDA** — El código se rescata directamente o con ajustes menores.
- **QUEDA (simplificar)** — Se rescata pero requiere refactor sustancial.
- **QUEDA (absorber)** — Se fusiona con otra feature (aplica solo a PWA).
- **QUEDA (reformular)** — La idea es correcta, la implementación se reescribe.
- **QUEDA (reemplazar)** — El concepto se conserva, se implementa desde cero.
- **ARCHIVAR** — El código se conserva en el repositorio pero no se activa en v2.
- **ELIMINAR** — El código se borra completamente del repositorio.

1. Core Governance (7 features)

#	Feature	Decisión	Justificación
1	Trees (CRUD, miembros, invites, visibilidad)	QUEDA	Workspace personal/equipo reutilizable. Se eliminan campos geo (country, city, lat/lon) y modo AI_COUNCIL.
2	Needs pipeline (Need → Idea → Branch → Task)	QUEDA (simplificar)	Needs se simplifican a título, descripción, puntos, status básico. Se eliminan: Ideas, quorum, relevance thresholds, sedimentation, importance voting.
3	Ramas hashtag (creación directa por admin)	QUEDA	Forma nativa de organizar IAs por especialidad en v2. <code>POST /api/branches/hashtag</code> con <code>#nombre</code> .
4	Branches normales (vía votación comunitaria)	ELIMINAR	Requieren Need → Idea → votación → quorum → 6 fases. Incompatible con v2 sin gobernanza democrática.
5			

#	Feature	Decisión	Justificación
	Tasks + asignación	QUEDA (simplificar)	Se conserva: crear/asignar/completar con evidencia + XP. Se elimina: difficulty voting, civic audit (20%), golden tickets, elite check, anonymous tasks.
6	Motor económico de puntos (1000 pts/mes)	QUEDA	Distribución proporcional por valorOficial × nivel. Intacto con ajustes menores.
7	Financing modes (GRATUITO/ FIJA/VARIABLE)	QUEDA (simplificar)	GRATUITO se conserva. SUSCRIPCIÓN pasa a ser a nivel plataforma (no por árbol). Se archiva: billing proporcional, cron de facturación, subscriptionService.

Archivos eliminados: consensusEngine.ts, ficheros de BranchNeedVote, PhaseDeliverable, billingCron.

Referencia: docs/audit-core-governance.md

2. People & Skills (9 features)

#	Feature	Decisión	Justificación
1	Auth (JWT, register/ login, guest-join)	QUEDA	JWT simple con guest-join cubre IAs + humanos. Sin cambios requeridos.
2	User / TreeMember	QUEDA	Soporte nativo de IAs (isAI, aiProfile, aiProvider, aiModel, aiOwnerId). TreeMember con skills JSON, xp, level.
3	XP + niveles (xp/50 + 1)	QUEDA	Fórmula meritocrática directa. IAs ganan XP completando tasks → suben de nivel. Perfecto para v2.

#	Feature	Decisión	Justificación
4	Skill XP / percentiles (UserSkillXP, cachedPercentile)	QUEDA	Esencial para v2. Separa XP general de skill XP. cachedPercentile permite escalar. Base del sistema de skills entrenables.
5	Expert Endorsements (avales entre miembros)	ARCHIVAR	Redundante en sistema de IAs puras. El XP por tarea ya certifica competencia. Código se conserva para posible reactivación si se añaden humanos.
6	Skill Influence System (votación ponderada green/golden)	ELIMINAR	v2 no tiene gobernanza compleja ni votación de necesidades. ~500 líneas a eliminar.
7	Career Path Network (grafo + Dijkstra)	QUEDA	Altísimo valor para v2. IAs planifican su propio crecimiento. Co-ocurrencia + transiciones temporales, 3 modos de ruta, Dijkstra multi-source.
8	Golden Tickets (7-task streak)	ARCHIVAR	Gamificación atractiva pero compleja. XP + niveles ya motiva suficientemente en v2. Código se conserva como módulo opcional futuro.
9	Privacy Settings (GDPR, visibilidad)	QUEDA (simplificar)	Simplificar a 2 niveles: TREE_ONLY y PUBLIC. Quitar TRUST_NETWORK y granularidad excesiva.

Archivos eliminados: skillInfluenceService.ts, skillInfluenceCron.ts, scoring.ts (parcial).

% código rescatable: ~70%

Referencia: docs/audit-people-skills-v2.md

3. Economy & Finance (9 features)

#	Feature	Decisión	Justificación
1	Need Points (1000/mes, congelamiento, renovación)	QUEDA	Core de priorización colectiva. Simplificar quitando multiplicador de concentración (>85% → ×2) y peopleEquivalent.
2	Sedimentation / Base Needs (12 meses → BASE)	ARCHIVAR	Implica 12 meses de maduración, 2 ciclos trimestrales, threshold 66%, degradación automática. Demasiado complejo para v2.
3	Fiat Ledger / Transactions (INCOME, EXPENSE, etc.)	ELIMINAR	v2 no tiene economía fiat interna. 7+ modelos de schema a eliminar.
4	Berries Economy (population threshold 10k)	ELIMINAR	Economía paralela compleja (reducción 10%/mes, P2P transfers). Nunca completamente implementada.
5	Trust Wallet (wallet unificado fiat+berries)	QUEDA (reemplazar)	Reemplazado por campo subscriptionTier + subscriptionExpiresAt en User. Sin balances monetarios.
6	TrustCore Fee Protocol (fee dinámico en P2P)	ELIMINAR	Sin economía fiat interna ni P2P, no aplica. 3+ modelos a eliminar.
7	Escrow Trust- less (graduated release, quorum 60%)	ELIMINAR	Sin economía interna, no hay pagos que poner en escrow. 3+ modelos a eliminar.

#	Feature	Decisión	Justificación
8	Payment providers (Khipu, Mercado Pago)	QUEDA (nuevo requisito)	No implementados hoy — solo existe slot <code>providerTxId</code> . Para v2 hosted se necesita integración real con Khipu.
9	Billing / Subscription (suscripción por árbol)	QUEDA (reformular)	La suscripción pasa de nivel TreeMember a nivel plataforma/ usuario. Máquina de estados ACTIVE → GRACE → SUSPENDED se rescata como base.

Modelos de schema eliminados (14): FiatTransaction, FiatTemplate, TreeExpense, BerryConfig, BerryTransaction, BerryMonthlyCycle, TrustCoreConfig, GlobalFeeConfig, FeeDistribution, UserWallet, WalletTransaction, MemberPayment, TreeTreasury, PromiseP2P.

Servicios eliminados (8): escrowService, disputeService, feeEngine, feeRecalculation, releaseCron, baseNeedReviewCron, billingCron, subscriptionCron.

% código rescatable: ~30%

Referencia: docs/audit-economy-finance-v2.md

4. AI & Automation (6 features)

#	Feature	Decisión	Justificación
1	Hermes Bridge (Task Matcher + AI Executor + Reputation + Asimov)	QUEDA (ampliar)	Núcleo de v2. IAs ganan XP completando tasks. Se amplía: modelos entrenables que mejoran con cada task.
2	Hermes Concierge (NL chat → crear árboles/ needs/ramas)	QUEDA (simplificar)	Útil para onboarding. Eliminar wizard de fases (TutorialPhase). Mantener chat NL simple.
3	AI Council Trees (parlamento de IAs, votación 1-10)	ARCHIVAR	Demasiado complejo. Duplica gobernanza humana. Depende de API keys externas

#	Feature	Decisión	Justificación
			(Moltbook). En v2 las IAs compiten por tasks, no votan.
4	AIMemberConfig (autonomyLevel, maxConcurrentTasks, etc.)	QUEDA (simplificar)	Colapsar autonomyLevel → booleano autoClaimEnabled . Eliminar aiFailCount/ aiRateLimitedUntil → reemplazar con quotaUsed/ quotaLimit.
5	Cron jobs (aiDailyInteraction, aiCouncilAudit)	ELIMINAR	aiDailyInteractionCron: 384 líneas, Rube Goldberg machine. En v2 las IAs están activas 24/7. aiCouncilAuditCron se va con AI Council.
6	Kanban integration (dispatcher → hermes kanban create)	QUEDA (cambios)	Cambiar execSync → spawn async. Si IAs corren como GGUF local, usar worker queue (BullMQ) en vez de Kanban CLI.

Archivos eliminados: aiCouncilController.ts (407 líneas), aiCouncilRoutes.ts, aiCouncilAuditCron.ts, aiDailyInteractionCron.ts (384 líneas), seed-ai-council.ts (277 líneas).

Nuevos componentes: modelRegistry, inferenceQueue, fineTunePipeline, billing routes, FineTuneSession, TrainingExample.

% código rescatable: ~65%

Referencia: docs/audit-ai-automation-v2.md

5. PWA Architecture (10 features)

#	Feature	Decisión	Justificación
1		ELIMINAR	v2 es UNA app. Flavor-gates (isTrustLite && ,

#	Feature	Decisión	Justificación
	Sistema multi-flavor (6 PWAs, Vite modes)		<code>isBranch0S &&)</code> son overhead puro. <code>.env.{flavor}</code> desaparecen.
2	Trust Lite (5173, dashboard principal)	QUEDA	El flavor más completo se convierte en LA app única. Absorbe todo lo demás.
3	Branch OS (5174, operaciones de rama)	QUEDA (absorber)	Su contenido (task board) se convierte en sección dentro de TreeDetail o Dashboard.
4	Trace Lite (5175, perfil/reputación)	QUEDA (absorber)	Componentes reutilizables (TraceProfile, TalentHunter, CareerPath) se mueven a Trust Lite.
5	Trust Insight (5176, radar cross-tree, métricas)	QUEDA (absorber)	Se convierte en ruta <code>/insight</code> o pestaña en Dashboard.
6	Trust Landing (5177, página pública)	ARCHIVAR	Se externaliza como sitio estático separado (HTML puro o Astro). No comparte codebase con la app principal.
7	Cross-PWA SSO (session-token, AppSwitcher)	ELIMINAR	Con 1 sola app, SSO entre PWAs es innecesario. Se elimina: AppSwitcher, crossLogin, ssoChecking.
8	PWA features (service workers, offline, install)	QUEDA (simplificar)	1 SW único en vez de 6. Cache-first para assets. 1 manifest con 2 íconos.
9	Vite preview vs dev (problemas con túneles mobile)	QUEDA	Vite sigue siendo la herramienta correcta. Con 1 app: 1 build, 1 preview port, 1 túnel.

#	Feature	Decisión	Justificación
10	MobileShell (matriz Crear/Hacer/Medir)	QUEDA (unificado)	Matriz unificada con 4 entidades: árbol, necesidad, rama, tarea.

Impacto cuantitativo: 6 PWAs → 1 SPA. 6 builds → 1 build. 12 puertos → 2 puertos. 6 túneles → 1 túnel.

% código rescatable: ~80%

Referencia: docs/audit-pwa-architecture-v2.md

6. Analytics & Export (8 features)

#	Feature	Decisión	Justificación
1	Public Metrics (GET /api/public/metrics)	QUEDA	Métricas ligeras de uso público. Valioso para transparencia open-source.
2	Tree Export JSON (GET /api/exports/tree/:id)	QUEDA	Exportación de datos de árbol en JSON. Simplificar quitando PDF.
3	EventLog simplificado (eventLogService.logEvent)	QUEDA (simplificar)	Se conserva el registro de eventos core. Se eliminan: controller HTTP, rutas REST independientes.
4	Health check (GET /api/ping)	QUEDA	Básico, necesario para monitoreo y deployments.
5	Business Metrics (métricas de negocio avanzadas)	ARCHIVAR	Métricas complejas de adopción/crecimiento. Solo relevantes si hay versión hosted con dashboard admin.
6		ELIMINAR	Dependencia pdfkit pesada. En v2 los

#	Feature	Decisión	Justificación
	PDF Export (pdfkit, generación de reportes)		reportes son markdown o via md2pdf externo.
7	Profile Export (exportación de perfiles)	ELIMINAR	Funcionalidad no esencial. Si se necesita en v2, reimplementar como JSON.
8	FinancialDashboard (frontend, gráficos recharts)	ELIMINAR	Dashboard financiero complejo con gráficos. v2 no tiene economía fiat que graficar.

Archivos eliminados: metricsController.ts, metricsRoutes.ts, pdfExportService.ts, eventLogController.ts, eventLogRoutes.ts, FinancialDashboard.tsx.

Endpoints eliminados: 9. Endpoints conservados: 4.

Líneas a remover estimadas: ~1,200 LOC.

Dependencias candidatas a eliminación: pdfkit, recharts.

% código rescatable: ~55%

Referencia: handoff kanban t_71a3c4ab

7. External/Enterprise (8 features)

#	Feature	Decisión	Justificación
1	External Needs (necesidades externas a árbol)	ELIMINAR	Pipeline enterprise completo que v2 no hereda. Implica matching cross-tree complejo.
2	External Candidates (candidatos externos)	ELIMINAR	Sistema de postulación externa con evaluación multi-evaluador anónima. Fuera de alcance v2.
3	Insight External Openings	ELIMINAR	Radar de oportunidades externas. Depende de External

#	Feature	Decisión	Justificación
			Candidates y Corporate Referrals.
4	Corporate Referrals (referencias corporativas)	ELIMINAR	Sistema de referencias entre empresas. Innecesario en v2.
5	Solution Proposals (propuestas de solución)	ELIMINAR	Pipeline de propuestas con budget fiat. Fuera de alcance.
6	Evaluator Dashboard (dashboard de evaluador)	ELIMINAR	Dashboard para evaluadores multi-árbol. Eliminado junto con External Candidates.
7	Insight Signals (radar cross-tree de necesidades)	ARCHIVAR (reimplementar)	Concepto valioso: detección de necesidades entre árboles. Se reimplementa en v2 simplificado sin pipeline enterprise.
8	Insight Internal Matches (matchScore IA-skills)	ARCHIVAR (reimplementar)	Concepto valioso: matching de skills de IAs con necesidades detectadas. Reimplementación limpia sin escalation pipeline ni budget fiat.

Líneas de schema afectadas: ~800 líneas. Rutas afectadas: 3.

% código rescatable: ~15% (conceptual, requiere reimplementación)

Referencia: handoff kanban t_78075a92

Lo que se queda

Features, componentes y código que se rescatan para Trust Lite v2, agrupados por dominio.

Core Governance (~2,112 LOC rescatable, 40%)

- **Trees:** CRUD completo, miembros, invites, visibilidad (pública/privada), admissionPolicy. Sin campos geo ni modo AI_COUNCIL.
- **Needs:** CRUD simplificado (título, descripción, puntos, status), motor de puntos mensual (1000/mes).
- **Ramas hashtag:** Creación directa por admin con `#nombre` , `isHashtag: true` , `type: HASHTAG` .
- **Tasks:** Crear, asignar, completar con evidencia. Status OPEN/IN_PROGRESS/COMPLETED. XP al completar.
- **Motor económico:** Distribución de presupuesto 1000 pts por valorOficial × nivel.
- **Financiamiento:** GRATUITO (default) + suscripción simple plataforma.

People & Skills (~70% rescatable)

- **Auth:** JWT, register, login, guest-join. Sin OAuth/social login (innecesario).
- **User / TreeMember:** Soporte nativo IA (isAI, aiProfile, aiModel, aiOwnerId). Skills JSON.
- **XP + Niveles:** Fórmula directa `xp/50 + 1` . Sin boost por endorsements.
- **Skill XP / Percentiles:** Tabla UserSkillXP con completedTasks + accumulatedPoints + cachedPercentile. Base del sistema de skills entrenables.
- **Career Path Network:** Grafo de co-ocurrencia de skills con Dijkstra. 3 modos: normal, inverso, multi-source. IAs planifican su crecimiento.
- **Privacy:** Simplificado a 2 niveles: TREE_ONLY y PUBLIC.

Economy & Finance (~30% rescatable)

- **Need Points:** Pool de 1000 puntos/mes, congelamiento al asignar, renovación. Sin multiplicador de concentración ni peopleEquivalent.
- **Payment providers:** Integración real con Khipu/MercadoPago (infraestructura nueva, concepto heredado).

- **Subscription:** Máquina de estados ACTIVE → GRACE → SUSPENDED rescutada, movida a nivel plataforma.

AI & Automation (~65% rescatable)

- **Hermes Bridge:** Task Matcher por skill overlap, AI Executor, AI Reputation, Protocolo Asimov H1–H5.
- **Concierge:** Chat NL para crear árboles/needs/ramas. Sin wizard de fases.
- **AIMemberConfig:** Simplificado: `autoClaimEnabled`, `maxConcurrentTasks`, `allowedPhases`, `skillOverrides`. Con nuevos campos: `modelPath`, `modelRepo`, `apiKey`, `quotaUsed`, `quotaLimit`.
- **Kanban integration:** Dispatch async vía worker queue (BullMQ).

PWA Architecture (~80% rescatable)

- **1 SPA React + Vite:** Colapso de 6 PWAs en una sola app con React Router v7.
- **MobileShell unificado:** Matriz Crear/Hacer/Medir con 4 entidades (árbol, necesidad, rama, tarea).
- **1 service worker, 1 manifest:** Simplificación de 6 a 1.
- **Componentes absorbidos:** BranchOSDashboard, TraceProfile, TalentHunter, CareerPath, TrustInsightDashboard, TrustWallet.
- **Landing page:** Externalizada como sitio estático separado.

Analytics & Export (~55% rescatable)

- **Public Metrics:** GET /api/public/metrics.
- **Tree Export JSON:** GET /api/exports/tree/:id.
- **EventLog:** `eventLogService.logEvent` (sin controller HTTP).
- **Health check:** GET /api/ping.
- **Dashboard admin hosted:** 5 métricas (IA activity, evaluation throughput, tree health, error rate, billing).

External/Enterprise (~15% rescatable, conceptual)

- **Insight Signals:** Concepto de radar cross-tree de necesidades. Reimplementación limpia en v2.

- **Insight Internal Matches:** Concepto de matching IA-skills con necesidades. Reimplementación sin pipeline enterprise ni budget fiat.
-

Lo que se va

Features, componentes y código a eliminar o archivar, con justificación.

ELIMINAR (código borrado del repositorio)

#	Feature	Dominio	LOC estimadas	Razón
1	Branches normales (votación, 6 fases)	Core Governance	~1,500	Sin gobernanza democrática en v2
2	Skill Influence System	People & Skills	~500	Votación ponderada innecesaria sin gobernanza
3	Fiat Ledger / Transactions	Economy	~1,200	Sin economía fiat interna
4	Berries Economy	Economy	~800	Economía paralela nunca completamente implementada
5	TrustCore Fee Protocol	Economy	~600	Sin economía P2P
6	Escrow Trust-less	Economy	~500	Sin pagos internos
7	AI Council (parlamento de IAs)	AI & Automation	~1,070	Demasiado complejo, duplica gobernanza
8			~384	

#	Feature	Dominio	LOC estimadas	Razón
	Cron jobs (aiDailyInteraction, aiCouncilAudit)	AI & Automation		IAs activas 24/7 en v2; Rube Goldberg
9	Sistema multi-flavor (6 PWAs)	PWA	~300	1 sola app en v2
10	Cross-PWA SSO (AppSwitcher, crossLogin)	PWA	~160	SSO innecesario con 1 app
11	PDF Export (pdfkit)	Analytics	~400	Dependencia pesada; reportes vía markdown
12	Profile Export	Analytics	~200	No esencial; reimplementar como JSON si se necesita
13	FinancialDashboard (recharts)	Analytics	~300	Sin economía fiat que graficar
14	External Needs/ Candidates/Openings	External	~600	Pipeline enterprise fuera de alcance
15	Corporate Referrals	External	~250	Sistema de referencias entre empresas
16	Solution Proposals	External	~300	Pipeline con budget fiat
17	Evaluator Dashboard	External	~200	Eliminado con External Candidates

#	Feature	Dominio	LOC estimadas	Razón
18	Business Metrics (complejas)	Analytics	~200	Solo relevantes con hosted; archivar no eliminar

Total líneas estimadas a eliminar: ~8,000–9,000 LOC

Modelos de schema eliminados: 14

Enums eliminados: 13

Dependencias eliminadas: pdfkit

ARCHIVAR (código se conserva, no se activa en v2)

#	Feature	Dominio	Razón para archivar (no eliminar)
1	Sedimentation / Base Needs	Economy	Puede reactivarse si se necesita antigüedad de necesidades
2	Expert Endorsements	People & Skills	Puede reactivarse si se añaden humanos que avalan IAs
3	Golden Tickets	People & Skills	Gamificación atractiva; simplificar a 1 ticket si se reactiva
4	AI Council Trees	AI & Automation	Concepto interesante de IAs evaluando necesidades
5	Trust Landing (PWA separada)	PWA	Reimplementar como sitio estático separado
6	Business Metrics	Analytics	Solo si hay versión hosted con dashboard admin
7	Insight Signals	External	Concepto valioso; reimplementación limpia en v2
8		External	

#	Feature	Dominio	Razón para archivar (no eliminar)
	Insight Internal Matches		Concepto valioso; reimplementación sin pipeline enterprise

Lo nuevo

Features, componentes y servicios que Trust Lite v2 necesita y que **no existen** en el código actual.

1. Modelos open-source descargables (GGUF + HuggingFace)

- **Formato:** GGUF cuantizados (Q4_K_M, Q5_K_M, Q8_0) vía llama.cpp
- **Distribución:** HuggingFace Hub (`huggingface.co/trustsuite/`)
- **Modelos base:** trust-lite-v2-base, trust-lite-v2-coding, trust-lite-v2-design
- **Config por modelo:** skills declarados, prompt template, context length, hardware requerido
- **Descarga:** Backend descarga GGUF vía `huggingface_hub` Python SDK
→ `/home/user/.trustsuite/models/<treeId>/<aiId>/`
- **Inferencia local:** llama.cpp server mode en puerto dinámico. Una instancia por IA activa.
- **Seguridad:** Firmado HMAC/SHA256 checksum desde HuggingFace

2. Fine-tuning pipeline (Unsloth QLoRA)

- **Framework:** Unsloth (2-5× más rápido, menos VRAM que vanilla LoRA)
- **Trigger:** Cada N tasks completadas exitosamente (configurable, default 10)
- **Dataset:** Pares (task.description → deliverable.output) por IA, formato ShareGPT/JSONL
- **Ciclo:** Buffer se llena → Unsloth QLoRA fine-tune sobre GGUF base → checkpoint LoRA mergeado → nuevo GGUF
- **Métricas:** Loss antes/después, win rate A/B test, XP gain post-fine-tune
- **Schedule:** Background, max_steps=200 (5-15 min en GPU)

3. API Keys + Billing (modelo de negocio)

Tiers propuestos:

Tier	Costo/ mes	IAs	Fine- tuning	Tasks/ día	Inferencia
Free	\$0	1 IA local	No	5	GGUF en tu hardware
Pro	\$15	3 IAs locales	Cada 10 tasks	Ilimitado	GGUF local
Cloud	\$30	3 IAs cloud	Cada 10 tasks	Ilimitado	Cloud (Groq/DeepSeek)

Endpoints nuevos: - `POST /api/ai/keys` — Crear API key - `GET /api/ai/usage` — Cuota consumida (tasks, tokens, fine-tune sessions) - `POST /api/ai/billing/webhook` — Webhook de pago confirmado (Khipu)

4. Model Registry Service

- Descarga de GGUF desde HuggingFace Hub
- Health check de modelos descargados
- Gestión de versiones de modelo por IA
- Fallback a cloud inference si no hay hardware local

5. Inference Queue Service

- Cola de requests a llama.cpp servers (BullMQ)
- Rate limiting por API key, no por contador de fallos
- Prioridad: Cloud > Pro > Free
- Timeout y reintentos automáticos

6. Suscripción a nivel plataforma

- **Modelo:** `Subscription` a nivel `User` con `subscriptionTier` (FREE, PRO, ENTERPRISE), `subscriptionExpiresAt`, `subscriptionStatus` (ACTIVE, GRACE, CANCELLED)

- **Integración Khipu:** URL de pago generada por backend, webhook de confirmación
- **Freemium:** 14 días de prueba PRO → degradación automática a FREE
- **Facturación:** Boletas electrónicas (SII Chile), historial de pagos, exportación de recibos

7. Dashboard admin hosted

- **Endpoint:** GET `/api/admin/metrics` (solo hosted)
- **Métricas:** IA activity (tasks/día), evaluation throughput, tree health, error rate, billing
- **Visualización:** Gráficos en `/admin` dentro de la SPA

8. Nuevos modelos de schema

Modelo	Propósito
<code>FineTuneSession</code>	Sesión de fine-tuning: aiMemberId, baseModel, loraPath, métricas, status
<code>TrainingExample</code>	Ejemplo de entrenamiento: aiMemberId, taskId, prompt, completion
<code>Subscription</code> (o campo en User)	Tier, expiresAt, status, paymentHistory
<code>ApiKey</code>	API keys por usuario con scopes y rate limits
<code>ModelMetadata</code>	Registro de modelos GGUF descargados y sus checksums

Recomendación Final: Plan de Acción por Fases

Fase 1: Limpiar (4–6 semanas)

Objetivo: Eliminar todo el código marcado ELIMINAR y ARCHIVAR, dejando solo el núcleo rescatable.

1. Semana 1–2: Schema + Backend

2. Eliminar 14 modelos del schema Prisma (FiatTransaction, Berries, FeeProtocol, Escrow, AI Council)
3. Eliminar 13 enums obsoletos
4. Eliminar archivos backend marcados (~25 archivos: controllers, services, utils, cron, routes)
5. Generar migración de Prisma
6. Verificar que el backend compila sin los archivos eliminados

7. **Semana 3–4: Simplificaciones**

8. Simplificar Needs pipeline (quitar Ideas, quorum, relevance thresholds)
9. Simplificar Tasks (quitar difficulty voting, civic audit, golden tickets, elite check)
10. Simplificar Financing (solo GRATUITO + suscripción simple)
11. Simplificar AIMemberConfig (autonomyLevel → autoClaimEnabled, quitar fail counters)
12. Simplificar Privacy (2 niveles: TREE_ONLY, PUBLIC)
13. Eliminar cron jobs (aiDailyInteraction, aiCouncilAudit, billingCron, etc.)

14. **Semana 5–6: Frontend**

15. Colapsar 6 PWAs → 1 SPA React + Vite
16. Eliminar flavor-gates, SSO, AppSwitcher
17. Unificar MobileShell con 4 entidades
18. 1 service worker, 1 manifest
19. Externalizar landing page
20. Ejecutar test suite completa

Entregable: Trust Suite con ~50% del código eliminado, compilando, con tests pasando.

Fase 2: Construir nuevo (8–12 semanas)

Objetivo: Implementar features nuevas exclusivas de v2.

1. **Semana 7–9: Modelos + Fine-tuning**

2. modelRegistry service: descarga GGUF desde HuggingFace
3. inferenceQueue service: cola de requests a llama.cpp servers (BullMQ)
4. fineTunePipeline service: Unsloth QLoRA automation
5. FineTuneSession + TrainingExample modelos

6. fineTuneCron: dispara fine-tuning cuando buffer lleno

7. **Semana 10–12: Billing + API**

8. Modelo Subscription a nivel User (tiers Free/Pro/Cloud)

9. Integración real Khipu (URL de pago, webhook)

10. API keys con rate limiting por tier

11. Endpoints: `/api/ai/keys` , `/api/ai/usage` , `/api/ai/billing/webhook`

12. Dashboard admin hosted (5 métricas)

13. **Semana 13–15: Reimplementaciones selectas**

14. Insight Signals v2: radar cross-tree simplificado

15. Insight Internal Matches v2: matching IA-skills sin pipeline enterprise

16. ConciERGE simplificado (sin wizard fases)

17. UI de fine-tuning y modelos en el frontend

Entregable: Trust Lite v2 funcional con IAs entrenables, billing, y modelos GGUF.

Fase 3: Lanzar (4–6 semanas)

Objetivo: Pulir, asegurar calidad, y publicar.

1. **Semana 16–17: QA + Documentación**

2. Testing exhaustivo de features nuevas

3. Documentación open-source: README, CONTRIBUTING, docs.gettrustsuite.com

4. Guía de self-hosting con docker-compose

5. Tutorial de fine-tuning de IAs

6. **Semana 18–19: Infraestructura + Landing**

7. Landing page estática (trustsuite.com)

8. CI/CD con GitHub Actions

9. Docker images en GitHub Container Registry

10. Modelos base en HuggingFace Hub

11. **Semana 20: Lanzamiento**

12. Publicación open-source (GitHub public)
13. Blog post + anuncio
14. Versión hosted alpha (early adopters)

Entregable: Trust Lite v2 open-source publicado, modelos en HuggingFace, landing en producción.

Resumen de decisiones por archivo (backend)

Archivos a ELIMINAR (~25 archivos)

```
backend/src/  
├─ controllers/  
│   ├── aiCouncilController.ts  
│   ├── expertEndorsementController.ts  
│   ├── metricsController.ts  
│   ├── eventLogController.ts  
│   ├── subscriptionController.ts  
│   └─ walletController.ts (1392 líneas)  
├─ services/  
│   ├── escrowService.ts  
│   ├── disputeService.ts  
│   ├── expertEndorsementService.ts  
│   ├── skillInfluenceService.ts  
│   ├── subscriptionService.ts (reformular)  
│   └─ baseNeedService.ts (archivar lógica)  
├─ utils/  
│   ├── consensusEngine.ts  
│   ├── feeEngine.ts  
│   ├── goldenTicketEngine.ts (archivar)  
│   └─ scoring.ts (parcial)  
├─ cron/  
│   ├── aiCouncilAuditCron.ts  
│   ├── aiDailyInteractionCron.ts  
│   ├── billingCron.ts  
│   ├── subscriptionCron.ts  
│   └─ feeRecalculation.ts
```



```
| └─ releaseCron.ts
| └─ baseNeedReviewCron.ts
| └─ skillInfluenceCron.ts
└─ routes/
    └─ aiCouncilRoutes.ts
    └─ metricsRoutes.ts
    └─ eventLogRoutes.ts
    └─ expertEndorsementRoutes.ts
    └─ subscriptionRoutes.ts
└─ scripts/
    └─ seed-ai-council.ts
```

Archivos a SIMPLIFICAR (refactor sustancial)

```
backend/src/
└─ controllers/
    └─ needController.ts (quitar pipeline Ideas, thresholds)
    └─ taskController.ts (quitar difficulty votes, civic audit, golden t
    └─ financingController.ts (solo GRATUITO + SUBSCRIPCION_FIXED)
    └─ userController.ts (quitar referencias a goldenTickets, avales)
    └─ privacySettingsController.ts (reducir a 2 niveles)
└─ utils/
    └─ economicEngine.ts (ajustes menores)
    └─ privacy.ts (simplificar niveles)
└─ services/
    └─ eventLogService.ts (sin controller HTTP)
```

Archivos que QUEDAN intactos

```
backend/src/
└─ controllers/
    └─ treeController.ts
    └─ branchController.ts (hashtag)
    └─ authController.ts
    └─ careerPathController.ts
    └─ exportController.ts (simplificar)
    └─ taskController.ts (parcial)
└─ services/
```

```
| └─ careerPathService.ts
| └─ monthlyNeedPointsService.ts
| └─ modelRegistry.ts (NUEVO)
└─ utils/
    └─ eliteCalculator.ts
└─ middleware/
    └─ authMiddleware.ts
└─ routes/
    └─ careerPathRoutes.ts
    └─ exportRoutes.ts (simplificar)
```

Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Regresiones al eliminar ~8,000 LOC	Alta	Medio	Test suite exhaustiva en F1; migrar tests antes de eliminar código
Fine-tuning costoso sin GPU	Media	Alto	Cloud fallback (Groq/ DeepSeek); modelo base ya funcional sin fine-tune
Complejidad de GGUF + llama.cpp server	Media	Alto	Empezar con 1 modelo base bien probado; abstraer inferenceQueue
Khipu integración frágil	Media	Medio	Mock completo en desarrollo; webhook con reintentos
Adopción open-source baja	Baja	Crítico	Documentación excelente, docker-compose trivial, landing atractiva
Seguridad de modelos GGUF	Baja	Alto	Firmado HMAC/SHA256 desde HuggingFace; checksum verification en modelRegistry

Conclusión

Trust Suite v1 es un sistema ambicioso con 57 features que cubren gobernanza, economía, IA, reputación y enterprise. Trust Lite v2 es su destilación: de las 57 features, 31 se quedan (24 directamente + 7 con cambios), 8 se archivan, 18 se eliminan. Aproximadamente la mitad del código es rescatable.

El movimiento estratégico es claro: **eliminar primero, construir después**. La Fase 1 (limpiar) puede ejecutarse de inmediato y produce un codebase ~50% más pequeño, más mantenible, y sin deuda del pipeline de gobernanza compleja. La Fase 2 introduce el diferenciador real de v2: IAs como modelos open-source descargables que mejoran vía fine-tuning al completar tareas. La Fase 3 lanza el producto como open-source con modelo de negocio hosted.

El resultado: un sistema más simple, más enfocado, y genuinamente innovador — paperclip meritocrático donde las IAs no son prompts estáticos, sino modelos que aprenden.

Fin del documento de síntesis.